

Streams

"Explique-moi la différence entre un **Stream** et une **Collection** en Java. Pourquoi on utiliserait l'un plutôt que l'autre ?

Collection → stocke des données en mémoire, tu peux les parcourir plusieurs fois, ajouter/supprimer des éléments.

Stream → ne stocke rien. C'est un pipeline de traitement qui décrit *comment* transformer des données. Deux points clés que l'interviewer attend :

1. **Lazy Evaluation (évaluation paresseuse)** Les opérations intermédiaires (**filter** , **map** , **sorted** ...) ne s'exécutent **pas** immédiatement. Elles attendent qu'une opération terminale (**collect** , **forEach** , **count** ...) soit appelée.

```
1 Stream<String> stream = List.of("Ali", "Bob", "Clara")
2   .stream()
3   .filter(s -> {
4       System.out.println("filter: " + s); // pas encore exécuté !
5       return s.length() > 2;
6   });
7 // Rien ne s'affiche ici
8
9 stream.collect(Collectors.toList()); // là ça s'exécute
```

2. **Un Stream s'utilise une seule fois**

```
1 Stream<String> s = list.stream();
2 s.collect(Collectors.toList()); // OK
3 s.collect(Collectors.toList()); // IllegalStateException !
```

3. **Stream peut exister sans Collection**

```
1 Stream.of("a", "b", "c"); // depuis des valeurs
2 Stream.iterate(0, n -> n + 1); // stream infini !
3 Stream.generate(Math::random); // stream infini !
```

"Quelles sont les opérations intermédiaires et terminales ? Cite-moi des exemples et dis-moi pourquoi cette distinction est importante."

● Opérations Intermédiaires

- **Retournent un Stream** → donc chaînables
- **Lazy** → ne s'exécutent pas tout de suite
- Exemples :

Méthode	Ce qu'elle fait
---------	-----------------

<code>filter(predicate)</code>	garde les éléments qui matchent
<code>map(function)</code>	transforme chaque élément
<code>sorted()</code>	trie
<code>distinct()</code>	supprime les doublons
<code>limit(n)</code>	garde les n premiers
<code>skip(n)</code>	saute les n premiers

● Opérations Terminales

- **Déclenchant** l'exécution du pipeline
- **Retournent autre chose** qu'un Stream (liste, int, void...)
- Le Stream est **consommé** après, inutilisable

Méthode	Ce qu'elle retourne
<code>collect(...)</code>	une Collection (List, Set...)
<code>forEach(...)</code>	void
<code>count()</code>	long
<code>findFirst()</code>	Optional
<code>anyMatch(...)</code>	boolean
<code>reduce(...)</code>	une valeur agrégée

Exercice pratique — Tu codes

Voici une liste d'entiers :

```
1 | List<Integer> nombres = List.of(5, 3, 8, 1, 9, 2, 7, 4, 6);
```

Écris un Stream qui :

1. Garde uniquement les nombres **pairs**
2. Les **multiplie par 2**
3. Trie dans l'**ordre décroissant**
4. Retourne une **List**

```
1 | List<Integer> nombres = List.of(5, 3, 8, 1, 9, 2, 7, 4, 6);
2 |
```

```

3 List<Integer> result = nombres.stream()
4   .filter(e -> e % 2 == 0)           // 8, 2, 4, 6
5   .map(e -> e * 2)                   // 16, 4, 8, 12
6   .sorted(Comparator.reverseOrder()) // 16, 12, 8, 4
7   .toList();
8
9 System.out.println(result); // [16, 12, 8, 4]

```

  Bonus à savoir pour l'entretien

`toList()` (Java 16+) retourne une liste **immutable**. Si t'as besoin de modifier la liste après :

```
1 | .collect(Collectors.toList()) // mutable
```

" `map()` vs `flatMap()` — c'est quoi la différence ? Donne-moi un cas concret où tu dois utiliser `flatMap()` plutôt que `map()` ."

Règle simple à retenir :

`map()` → 1 élément donne **1 élément** `flatMap()` → 1 élément donne **plusieurs éléments** (aplatit le résultat)

 `map()` — transformation simple

```

1 List<String> noms = List.of("Ali", "Bob", "Clara");
2
3 List<Integer> longueurs = noms.stream()
4   .map(n -> n.length()) // chaque String → un Integer
5   .toList();
6
7 // [3, 3, 5]

```

1 élément entrant → 1 élément sortant. Simple.

 Le problème que `flatMap()` résout

Imagine que tu as **une liste de listes** :

```

1 List<List<Integer>> groupes = List.of(
2   List.of(1, 2, 3),
3   List.of(4, 5),
4   List.of(6, 7, 8, 9)
5 );

```

Avec `map()` tu obtiens... une liste de listes :

```

1 // Stream<List<Integer>> → pas ce qu'on veut
2 groupes.stream()
3   .map(g -> g.stream()) // Stream<Stream<Integer>>

```

Avec `flatMap()` tu **aplatis** en un seul Stream :

```

1 List<Integer> tousLesNombres = groupes.stream()
2   .flatMap(g -> g.stream()) // aplatit tout

```

```
3     .toList();
4
5 // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

"Explique-moi ce que fait `reduce()` et à quoi ça sert. C'est quoi la différence avec `collect()` ?"

L'idée centrale

`reduce()` **combine** tous les éléments du Stream en **une seule valeur**.

C'est ça la différence avec `map()` qui **transforme** chaque élément individuellement.

La signature

```
1 | reduce(valeurInitiale, (accumulateur, elementCourant) -> résultat)
```

`reduce()` `collect()` Retourne **une valeur** (int, String...) **une Collection** (List, Map...)

Usage calculs, agrégations regrouper des éléments Exemple somme, max, concaténation
`toList()`, `toMap()`, `groupingBy()`

"Explique-moi ce que fait `collect()` et cite-moi les **Collectors** que tu connais avec leurs cas d'usage."

L'idée centrale

`collect()` est une opération **terminale** qui **regroupe** les éléments du Stream dans un conteneur (List, Set, Map, String...)

C'est le plus utilisé en pratique !

Les Collectors les plus importants

1 `toList()` / `toSet()`

```
1 | List<String> liste = stream.collect(Collectors.toList());
2 | Set<String> set = stream.collect(Collectors.toSet());
```

2 `joining()` — concaténer des Strings

```
1 | List<String> mots = List.of("Java", "est", "cool");
2 |
3 | String result = mots.stream()
4 |     .collect(Collectors.joining(", "));
5 | // "Java, est, cool"
6 |
7 | // Avec préfixe et suffixe :
8 | String result2 = mots.stream()
9 |     .collect(Collectors.joining(", ", "[", "]"));
10 | // "[Java, est, cool]"
```

3 counting() — compter

```
1 long count = stream.collect(Collectors.counting());
```

4 groupingBy() — grouper par critère très demandé

```
1 List<String> noms = List.of("Ali", "Adam", "Bob", "Clara", "Chloe");
2
3 Map<Character, List<String>> parLettre = noms.stream()
4     .collect(Collectors.groupingBy(n -> n.charAt(0)));
5
6 // {A=[Ali, Adam], B=[Bob], C=[Clara, Chloe]}
```

5 partitioningBy() — diviser en 2 groupes

```
1 List<Integer> nombres = List.of(1, 2, 3, 4, 5, 6);
2
3 Map<Boolean, List<Integer>> partition = nombres.stream()
4     .collect(Collectors.partitioningBy(n -> n % 2 == 0));
5
6 // {false=[1, 3, 5], true=[2, 4, 6]}
```

6 toMap() — convertir en Map

```
1 List<String> noms = List.of("Ali", "Bob", "Clara");
2
3 Map<String, Integer> nomLongueur = noms.stream()
4     .collect(Collectors.toMap(
5         n -> n,           // clé
6         n -> n.length()  // valeur
7     ));
8 // {Ali=3, Bob=3, Clara=5}
```